

HelixCode

HelixCode

The distributed AI development platform that divides work, preserves it, and never loses your place.

Summary

HelixCode is an enterprise-grade, Go-based distributed AI development platform that splits development work into intelligently-divided tasks across a network of SSH-managed workers, with automatic checkpointing and rollback so no work is ever lost. It unifies multi-provider LLM integration, full development-lifecycle workflows, and cross-platform delivery behind REST, CLI, TUI, and MCP interfaces.

Short description

HelixCode is a distributed AI development platform written in Go. It divides work into intelligent tasks across SSH-based worker networks, preserves progress with automatic checkpointing and rollback, integrates multiple LLM providers, and drives the full development lifecycle through REST, CLI, TUI, and MCP interfaces.

Long description

HelixCode is an enterprise-grade distributed AI development platform (`dev.helix.code`, MIT) built around a simple promise its tagline makes literal: divide the work, preserve it, and never lose your place. It is designed for intelligent task division, automatic work preservation, and cross-platform development workflows, and it is written in Go for the concurrency and single-binary portability that distributed computing demands — with automatic checkpointing, rollback, and real-time monitoring as first-class primitives rather than optional add-ons.

Its architecture layers a REST + WebSocket + MCP API surface over a set of focused core services — JWT authentication and session management, SSH-based worker-pool management with health monitoring, task management with checkpointing and dependency handling, project and workflow management, and a unified LLM provider layer — all persisted on PostgreSQL, with Redis available as an optional coordination and caching tier. Distributed workers auto-install across a network, so scaling out the fleet is a matter of pointing the server at a machine rather than hand-provisioning it, and multi-client interfaces span CLI, terminal UI, REST, and mobile frameworks so the same platform is reachable from a script, a terminal, or an app.

HelixCode drives a complete development lifecycle end to end: planning, building, testing, and refactoring workflows execute automatically with dependency awareness and multi-session context tracking, so a long-running effort keeps its thread across interruptions and machine boundaries. It integrates multiple LLM providers — llama.cpp, Ollama, and OpenAI — behind one interface, then adds hardware-aware model selection that detects available CPU/GPU/memory and matches the model to the machine, and supports advanced reasoning strategies such as chain-of-thought and tree-of-thoughts for problems that need more than a single pass. The Model Context Protocol is implemented across multiple transports for standardized tool and context exchange, and multi-channel notifications (Slack, Discord, Email, Telegram) keep teams informed as distributed work progresses. It targets Linux, macOS, Windows, Aurora OS, and SymphonyOS.

Why we built it

Distributed and AI-assisted development typically loses context and progress when tasks are split across machines or interrupted. HelixCode was built to make task division intelligent and work preservation automatic — so a large development effort can be broken down, distributed across a worker network, checkpointed, and resumed or rolled back without losing state.

Why it's a game-changer

It makes distributed AI development *durable* — the capability that was never practical when teams stitched these pieces together by hand. Three things that normally live in three separate tools become one platform: distributed compute (SSH worker networks with auto-installation and health monitoring), AI development assistance (multi-provider LLMs with reasoning and tool calling), and full lifecycle workflow automation. The connective tissue is database-backed checkpointing: because task state, checkpoints, and dependencies are persisted in PostgreSQL, a job that spans

many machines and many sessions can be rolled back or resumed exactly where it stopped. Interruptions and split work stop being a source of lost progress and become a routine, recoverable event.

What's innovative

- Work preservation as a core primitive: automatic checkpointing and rollback applied to *distributed* development tasks, so progress survives interruption and machine failure instead of evaporating with it.
- Hardware-aware model selection that inspects detected CPU/GPU/memory and matches each task to a model the machine can actually run well — no manual per-worker tuning.
- One platform, five front doors: REST, WebSocket, CLI, TUI, and MCP, with MCP itself exposed over multiple transports so tools and agents can integrate however they connect.
- Cross-platform reach that goes past the usual desktop trio to include Aurora OS and SymphonyOS, widening the worker fleet to platforms most tools ignore.

Biggest technical challenges & how we solved them

- **Not losing work across distributed, interruptible tasks.** When a job is split across machines, any crash or interruption normally strands whatever was in flight. We modeled the task itself as a carrier of checkpoints and dependencies, persisted in PostgreSQL, so the system can roll back to the last good state or resume from it — durability that lives in the data layer rather than in fragile in-memory state.
- **Managing a heterogeneous worker fleet.** A network of Linux, macOS, Windows, Aurora, and SymphonyOS machines is a moving target of availability and setup. We handle it with a dedicated worker-pool service that does SSH-based registration, auto-installation onto new nodes, and continuous health monitoring, so the fleet stays known and controllable as machines come and go.
- **Provider and hardware heterogeneity.** LLM backends and the machines that run them vary wildly in capability. We hid that behind a unified LLM provider interface and paired it with hardware detection (CPU/GPU/memory) that drives intelligent model selection, so the right model lands on the right machine without the caller having to reason about either.

Tech stack

- **Go (1.26+ inner module)** — chosen because its goroutine-based concurrency and single-binary output are exactly what a distributed worker system needs: cheap parallelism for orchestration and a self-contained binary that auto-installs onto any node. It carries all core services and the CLI/server binaries.
- **Gin (HTTP framework)** — chosen for a fast, minimal REST layer with low overhead; it serves the `/api/v1` surface (auth, workers, tasks, projects) that every client talks to.
- **PostgreSQL 15+ (via pgx/v5)** — chosen as the durable system of record because checkpointing and rollback demand transactional persistence; it holds the 11-table distributed-computing schema (users, workers, tasks, projects, sessions, llm_providers, notifications) that makes work preservation possible.
- **Redis 7+ (optional, go-redis/v9)** — chosen as an optional caching and coordination tier that speeds hot paths without becoming a hard dependency, so a minimal deployment still runs on Postgres alone.
- **SSH** — chosen as the worker-control transport precisely because it is already everywhere and already secure; it drives worker registration, auto-installation, and remote command execution across the whole pool with no bespoke agent to deploy first.
- **Model Context Protocol (MCP)** — chosen for standardized tool and context exchange so external tools and agents integrate through one open protocol; implemented with multi-transport support to meet clients wherever they connect.
- **LLM providers (llama.cpp, Ollama, OpenAI)** — chosen to span both local and hosted inference behind one unified interface, so hardware-aware selection can route a task to a local model or a hosted one without the caller knowing the difference.

Status & honesty notes

- **Status: beta.** The README self-reports a “FULLY COMPLETE / all 5 phases” state; that completeness is project-stated rather than independently verified, so the page treats it as beta.
- All specifics above trace to the repository README; marketing phrasings (taglines) are editorial rather than source metrics.

Priority tier: Helix-primary.